CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 27: Tool use and JSON schema enforcement

> **Exam objective:** Implement tool use with defined JSON schemas to enforce structured output compliance, and configure tool_choice to guarantee tool invocation when conversational responses would cause downstream failures.

Q1.

An orchestration service expects a model to emit a valid purchase-order object. When users ask casual questions, the model sometimes replies conversationally instead of producing JSON, breaking the parser. What is the best fix?

A. Increase temperature so the model is more creative with formatting B. Use tool calling with a defined JSON schema and set `tool_choice` so the tool must be invoked C. Add "please answer in JSON" to the system prompt and rely on compliance D. Strip non-JSON text with regex before parsing

Answer: B **Explanation:** Tool calling with a schema gives the model a structured contract, and forced `tool_choice` prevents fallback conversational responses that would break automation. Prompt-only and regex approaches are brittle and do not guarantee compliance.

Q2.

A pipeline extracts shipping addresses. The downstream service rejects payloads if `postal_code` is missing or if extra fields appear. Which setup provides the strongest enforcement?

A. Ask for a bullet list and transform it later B. Use a tool schema specifying required fields and allowed properties only C. Accept natural language and rely on a validation library after the fact D. Use XML instead of JSON

Answer: B **Explanation:** A JSON schema within tool use constrains both required structure and allowable fields before data reaches the downstream service. Post-validation can catch errors but does not prevent the model from producing unusable responses.

Q3.

Your application always needs a call to `submit_review_result` because a human-readable paragraph would be ignored by the processing queue. Which `tool_choice` strategy is best?

A. `auto`, because the model usually knows when a tool is needed B. Force `submit_review_result` so the model cannot answer conversationally C. Disable tools and parse the assistant text D. Let the

user decide whether the tool should be called

Answer: B **Explanation:** When downstream behavior depends on tool invocation, forced `tool_choice` is the correct control. `auto` may work often but still allows failures on edge cases, which is unacceptable in a strict pipeline.

Q4.

A team defines a tool schema for support ticket triage, but leaves `priority` as a free-form string. Outputs vary between `P1`, `High`, `urgent`, and `critical`. What is the best improvement?

A. Keep the field open-ended so the model can express nuance B. Replace `priority` with an enum of accepted values used by the ticketing system C. Ask the model to be more consistent in the prompt D. Convert the value to lowercase after parsing

Answer: B **Explanation:** When the downstream system expects a controlled set, the schema should encode that constraint explicitly with an enum. Prompt wording and post-processing can reduce variation but do not enforce correctness as strongly.

Q5.

A model must populate `scheduled_callback_time` only when the customer requested one. What is the best schema design for the tool?

A. Make the field required and set it to `"as soon as possible"` otherwise B. Make the field optional or nullable and document when it should be omitted C. Replace it with a paragraph field describing the request D. Require a timestamp in every case to simplify parsing

Answer: B **Explanation:** Conditional fields should be modeled explicitly as optional or nullable. Requiring a fabricated timestamp or replacing structured data with prose undermines automation and accuracy.

Q6.

Your system logs show that the assistant occasionally emits valid JSON in plain text instead of a tool call. The application ignores those outputs because only tool events are consumed. Which change best addresses the root cause?

A. Add a second parser for assistant text B. Force the tool with `tool_choice` and remove any ambiguity about acceptable response paths C. Lower max_tokens so the tool call is shorter D. Ask users to phrase requests more formally

Answer: B **Explanation:** The problem is not JSON syntax but response-channel selection. Forced tool invocation ensures the result arrives in the machine-consumable pathway the application actually processes.

Q7.

Why is a defined tool schema generally more reliable than asking for JSON in natural language?

A. The model treats schemas as a stronger structural contract than ordinary prose instructions B. Schemas always reduce token usage to zero C. Natural language cannot describe JSON keys accurately D. Tools bypass all model reasoning and directly query the database

Answer: A **Explanation:** Tool schemas formalize field names, types, and constraints, which increases structured compliance compared with prompt-only formatting requests. They do not eliminate reasoning or token use, but they reduce ambiguity significantly.

Q8.

An order-processing tool requires `currency` to be one of `USD`, `EUR`, or `JPY`. A developer instead accepts any string and plans to normalize later. What is the main risk?

A. None, because normalization later is always equivalent B. The model may emit unsupported values that pass initial parsing but fail business rules deeper in the system C. The response will no longer be valid JSON D. The model will refuse to answer when no enum is present

Answer: B **Explanation:** Loose schemas allow structurally valid but semantically invalid outputs to slip into the pipeline. Encoding business constraints in the schema catches problems earlier and reduces downstream failure modes.

Q9.

A compliance review tool has fields `finding`, `severity`, and `evidence`. Analysts complain that the model sometimes returns entire essays inside `finding`. What schema change helps most?

A. Add a maximum length expectation and clearer field descriptions within the schema or tool instructions B. Remove the `finding` field entirely C. Replace the tool with free-form chat D. Set temperature to 2.0

Answer: A **Explanation:** Rich field descriptions and bounded expectations guide the model toward the intended granularity while preserving structure. Removing the field or abandoning tool use would reduce utility, and temperature changes do not solve schema ambiguity.

Q10.

A workflow must call exactly one of two tools: `create_incident` or `answer_general_question`. In a specific endpoint, only incident creation is safe because unstructured answers are discarded. What is the best configuration?

A. Provide both tools and leave `tool_choice` on `auto` B. Force `create_incident` for that endpoint C. Disable all tools to simplify the prompt D. Ask the model to explain why a tool was not used

Answer: B **Explanation:** If only one response path is operationally valid, force that tool. Leaving both tools available with `auto` creates avoidable nondeterminism that can surface as dropped work.

Q11.

A developer includes a required field `resolution_code` in a tool schema, but many cases are still under investigation. The model invents placeholder codes to satisfy the schema. What is the best

remedy?

A. Remove the tool entirely B. Make `resolution_code` optional or nullable if unresolved cases are legitimate C. Keep it required and retrain users to ask better questions D. Replace the field with a boolean `done`

Answer: B **Explanation:** Required fields must reflect true business invariants. If a case can legitimately lack a resolution code, the schema should encode that; otherwise the model is incentivized to fabricate compliant-looking values.

Q12.

Which scenario most strongly justifies forced `tool_choice` instead of relying on `auto`?

A. The tool output is optional and a conversational summary is equally useful B. The UI displays either tool output or assistant text without issue C. Downstream automation fails completely unless a specific tool event is emitted every time D. The user prefers shorter responses

Answer: C **Explanation:** Forced `tool_choice` is warranted when tool invocation is not merely preferable but mandatory for correctness. If conversational output is acceptable, `auto` may be sufficient.

Q13.

A team wants the model to return a list of extracted line items. Sometimes there are no items. Which schema design is best for the tool result?

A. Return a paragraph saying "no items found" B. Use an array field, with empty array or null semantics clearly defined for no items versus unknown extraction status C. Force at least one placeholder object in the array D. Convert the array into a newline-delimited string

Answer: B **Explanation:** Arrays are the proper structured representation for repeated items, and explicit semantics for empty versus null preserve meaning. Placeholder objects and prose responses create downstream ambiguity and parsing burden.

Q14.

A model is calling the correct tool, but the payload often includes undeclared keys like `confidence_commentary`. How should you improve reliability?

A. Expand the parser to accept arbitrary keys B. Tighten the schema and descriptions so only supported properties are emitted, then validate tool payloads strictly C. Ignore extra keys because JSON is flexible D. Convert every payload to text before storage

Answer: B **Explanation:** Undeclared properties signal that the tool contract is underspecified or not being enforced tightly enough. Strong schemas plus strict validation protect downstream systems and keep outputs predictable.

Q15.

An engineering team says, "We already validate after the response, so tool schemas are unnecessary." What is the best rebuttal?

A. Post-validation is enough; schemas add no real value B. Schemas reduce invalid outputs at generation time, while post-validation only detects failures after latency and compute have already been spent C. Schemas are only useful for small prompts D. Validation libraries cannot process JSON from LLMs

Answer: B **Explanation:** Generation-time constraints and post-generation validation serve different purposes. Schema-guided tool use lowers the rate of bad outputs before they hit application code, while validation alone only catches errors after they occur.